

Object-Oriented Programming

@author Yangxin Wu

Last edited: 20251124

- [面向对象的三大特点](#)
 - [封装](#)
 - [继承](#)
 - [多态](#)
- [面向对象的七个原则](#)
 - [单一职责原则](#)
 - [开闭原则](#)
 - [里氏代换原则](#)
 - [依赖倒转原则](#)
 - [合成复用原则](#)
 - [接口隔离原则](#)
 - [迪米特法则](#)
- [UML 类图](#)
 - [类和接口的表示](#)
 - [类图中关系的表示](#)
- [设计模式](#)
 - [单例模式](#)
 - [工厂方法模式](#)
 - [抽象工厂模式](#)
 - [迭代器模式](#)
 - [策略模式](#)
 - [数据访问对象模式](#)
 - [观察者模式](#)
 - [模板方法模式](#)

面向对象的三大特点

所谓面向对象，即：

- 任何系统都是由能够完成一组相关任务的对象构成。
- 如果对象依赖于一个不属于它负责的任务，那么就需要访问负责此任务的另一个对象。
- 一个对象不能直接操作另一个对象内部的数据，它也不能使其它对象直接访问自己的数据。
- 所有的「交流」都是通过「方法」的调用完成。

封装

- 隐藏对象的属性和实现细节，仅对外公式访问方法。
- 增加安全性和简化编程。

继承

- 子类继承父类的特征和行为。
- 实现代码的复用。

多态

- 同一个行为具有不同表现性态的能力（「一个接口，多个方法」）。
- 提高了程序的扩展性和可维护性。

面向对象的七个原则

在设计面向对象的程序时，或者说，安排程序中的类和对象时，我们需要遵循七个原则。

单一职责原则

就一个类而言，应该仅有一个能引起它变化的原因。

- 不同变化 $\Rightarrow$ 不相干的职责 $\Rightarrow$ 难以复用。
- 如果一个类有多于一个的职责，考虑职责的分离。

开闭原则

一个软件实体应「对扩展开放，对修改关闭」：

- 通过抽象类和接口，规定了具体类的特征，实现「对修改关闭」。
- 利用重载改变具体类，可以改变系统的行为，实现「对扩展开放」。

例如，有下面的这些「水果」类：

```
class Fruit { ... }           // 父类，水果
class Apple extends Fruit { ... } // 子类，苹果
class Orange extends Fruit { ... } // 子类，橙子
class Banana extends Fruit { ... } // 子类，香蕉
```

如果想要引入一款新的电脑，不需要动上述任何代码（所谓的「闭」），只要增加（所谓的「开」）

```
class Passionfruit extends Fruit { ... } // 子类，百香果
```

里氏代换原则

使用父类的地方一定可以换成子类，反之不然。

```
Fruit a = new Orange();    // 没问题
Orange b = new Orange();   // 没问题
Orange c = new Fruit();    // 大问题
```

依赖倒转原则

高层模块不要依赖低层模块，而应该使用接口。

接上面水果的例子，若定义用户的「吃水果」函数为

```
class User {
    public void eatFruit(Orange obj) { ... }
}
```

那么用户就和 `Orange` 这一个类绑定了，后续更换变得麻烦。这就是「高层模块依赖低层模块」。

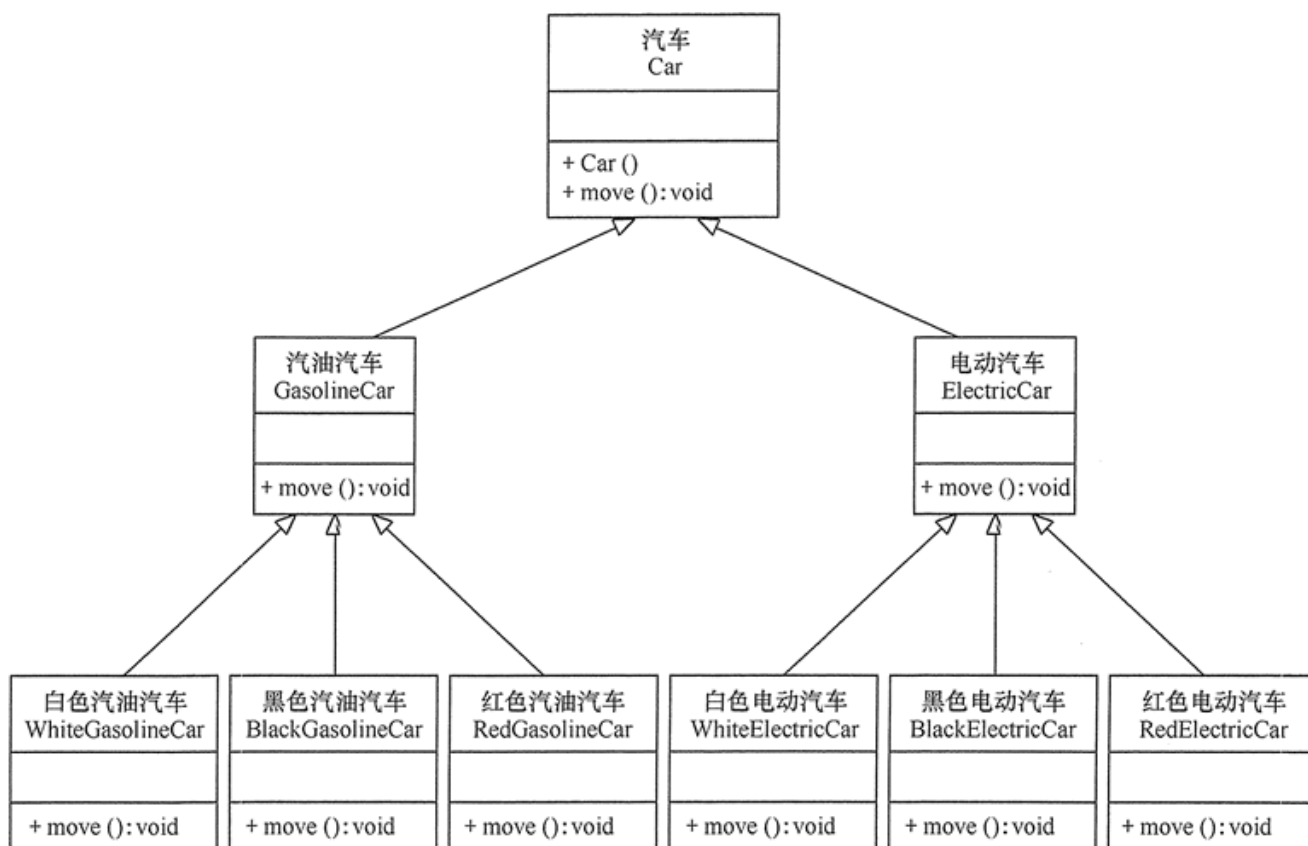
更好的写法应是

```
class User {
    public void eatFruit(Fruit obj) { ... }
}
```

合成复用原则

尽量使用对象组合，而不是继承来达到复用的目的。即：在一个新的对象里面 **使用**（聚合关系）一些已有的对象，使之成为新对象的一部分。

例如：汽车 `Car` 按动力划分可以分为油车、电车，按颜色划分可以分成黑车、白车、红车等。如果用继承的方法实现它们，组合种类会非常多：



反过来，如果将「发动机」 Engine、「颜色」 Color 作为 Car 的内部字段，则能大大地降低代码的复杂度。这就是合成复用。

接口隔离原则

客户端不应该依赖它不需要的接口。换句话说，使用多个专门的接口比使用单一的总接口要好。一旦一个接口太大，则需要将它分割成一些更细小的接口。

例如，定义下面的接口「日常生活」

```
interface DailyLife {
    public void sing();
    public void move();
    public void think();
    public void mate();
    ...
}
```

然后让 Person 类去实现它。这样造成的问题是，智能手机也能 sing 和 think，但不能 move 和 mate。那么，当我们为「智能手机」编写相关的类时，就得重新做一个接口「智能手机的日常生活」。解决方案是将这个大接口拆分：

```
interface Singable {
    public void sing();
}

interface Moveable {
    public void move();
}
```

```
}

interface Thinkable {
    public void think()
}

interface Mateable {
    public void mate();
}
```

因为 Java 中一个类可以实现多个接口，所以

```
class Person implements Singable, Moveable, Thinkable, Mateable { ... }
class Phone implements Singable, Thinkable { ... }
```

迪米特法则

一个软件实体应当尽可能少的与其他实体发生相互作用。每一个软件单位对其他的单位都只有最少的知识，而且局限于那些与本单位密切相关的软件单位。

UML 类图

统一建模语言（Unified Modeling Language, [UML](#)）是一种设计软件蓝图的可视化建模语言。

使用 UML 绘制的类与类之间的关系图称为类图。

类和接口的表示

- + 表示 public ;
- - 表示 private ;
- # 表示 protected ;
- 不带符号表示 friendly。

类图中关系的表示

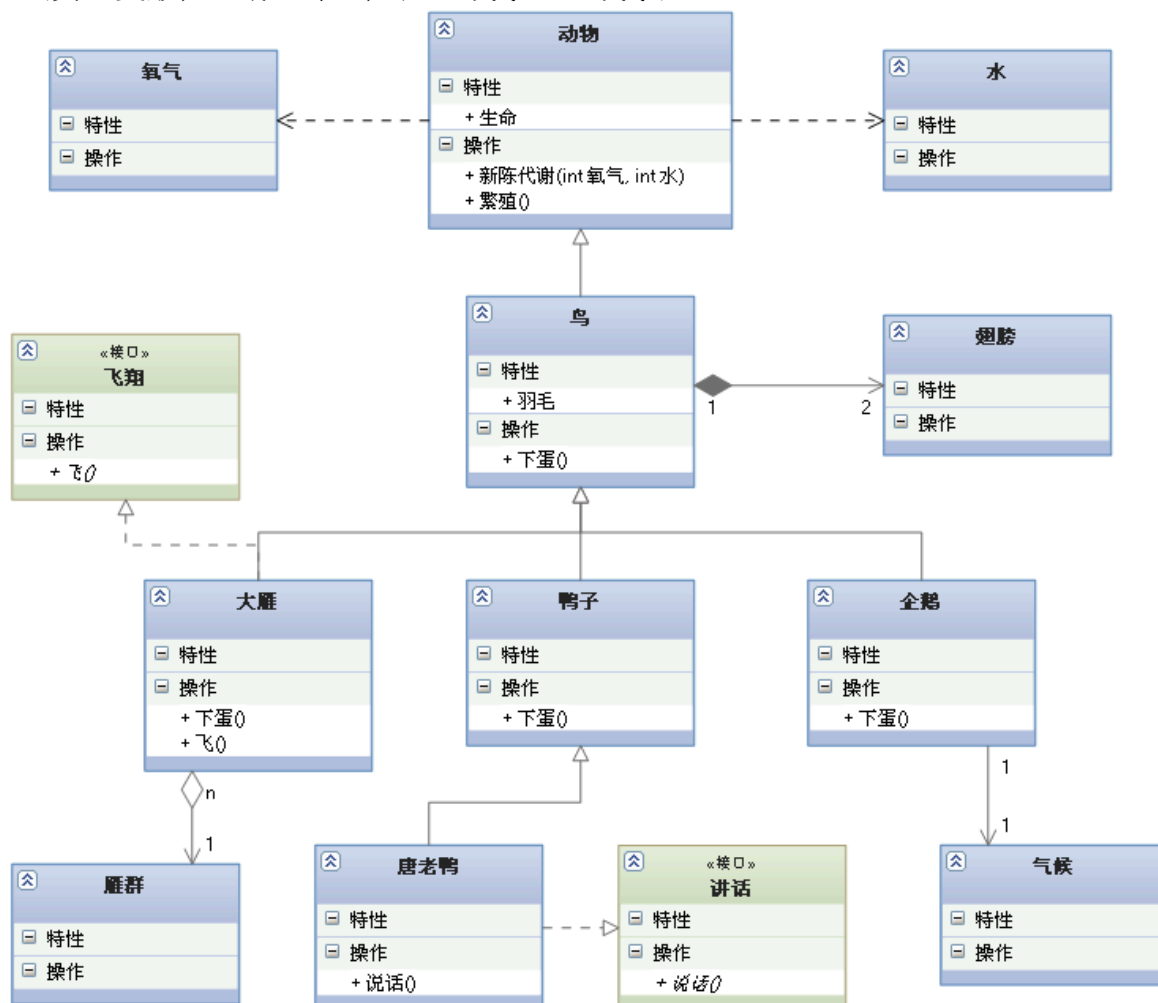
参见 <http://www.uml.org.cn/oobject/201609062.asp>

- 泛化（三角箭头实线，即继承 extends）
- 实现（三角箭头虚线，即实现 implements）
- 关联（普通箭头实线，A 与 B 有关系，如妻子与丈夫）

- 依赖（普通箭头虚线，A 依赖 B，如动物和氧气）

依赖关系的类型	关键字或构造型	描述
抽象	«abstraction»、«derive»、«refine» 或 «trace»	使在不同抽象级别或者从不同视点来表示同一概念的两个模型元素或多组模型元素相关
绑定	«bind»	将模板自变量连接至模板参数以根据模板来创建模型元素
实现	«realize»	指示客户模型元素是供应者模型元素的实现，而供应者模型元素是规范
替换	«substitute»	指示客户模型元素替代供应者；客户模型元素必须遵照供应者模型元素建立的合同或接口
使用	«use»、«call»、«create»、«instantiate» 或 «send»	指示一个模型元素需要另一个模型元素才能完全实现或执行操作

- 聚合（空心菱形，A 有一个 B，但 A 没了 B 也可以有）
- 组合（实心菱形，A 有一个 B，但 A 没了 B 也没了）



设计模式

一套被反复使用、经过分类编目的代码设计经验的总结。

设计模式根据「模式是用来完成什么工作的」来分，可以分为「创建型模式」「结构型模式」和「行为型模式」三种；根据「模式应用于类还是对象」，可以分为「类模式」和「对象模式」两种。

单例模式

使用静态变量，保证一个类仅有一个实例，并提供一个访问它的全局访问点。
实现对唯一实例的受控访问，减少了资源的开销，并能全局访问，避免对共享资源的多重占用。
单例模式允许可变数目的实例。

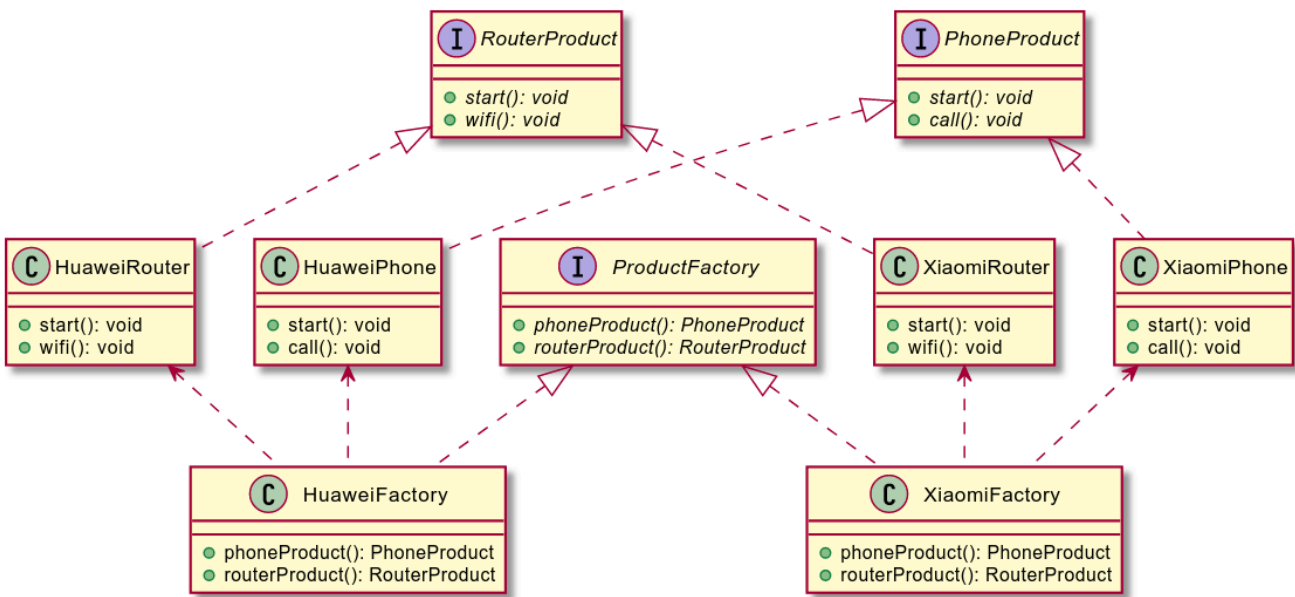
工厂方法模式

- 抽象工厂（抽象类） `Creator`：提供创建工厂的接口，指导「工厂」的结构。
 - 具体工厂 `ConcreteCreatorA`、`ConcreteCreatorB`：继承 `Creator` 并具体化「生产」不同产品的方法。
- 抽象产品（接口） `Product`：提供产品的功能接口，指导「产品」的结构。
 - 具体产品 `ConcreteProductA`、`ConcreteProductB`：实现 `Product` 并具体化不同的功能。

抽象工厂模式

适用于有多个产品族的情景。
围绕一个超级工厂创建其他工厂。该超级工厂又称为其他工厂的工厂。

使用这种模式，添加「产品等级」（在下例中，为增加一个手机品牌，如「三星」）相当简单，只要让工厂实现 `ProductFactory` 接口，让自己的产品分别实现 `PhoneProduct` 和 `RouterProduct` 接口就可以。但要增加产品族（下例中，为增加一个新的产品类型，如「平板电脑」）十分困难。

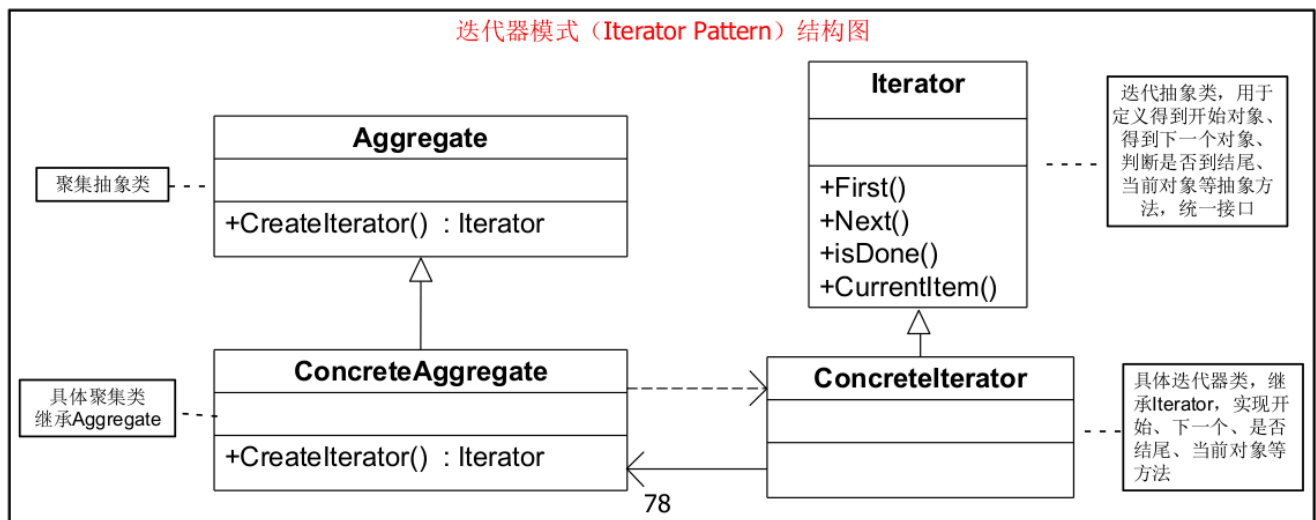


迭代器模式

对聚合类（例如 `List` 和 `Set`）的一种访问方式。
使用 `iterator()` 方法可以从支持迭代器模式的聚合类中获得迭代器 `Iterator`。
对迭代器有下面的常用方法：

- `next()` 获取下一个元素。
- `hasNext()` 检测集合中是否还有元素。

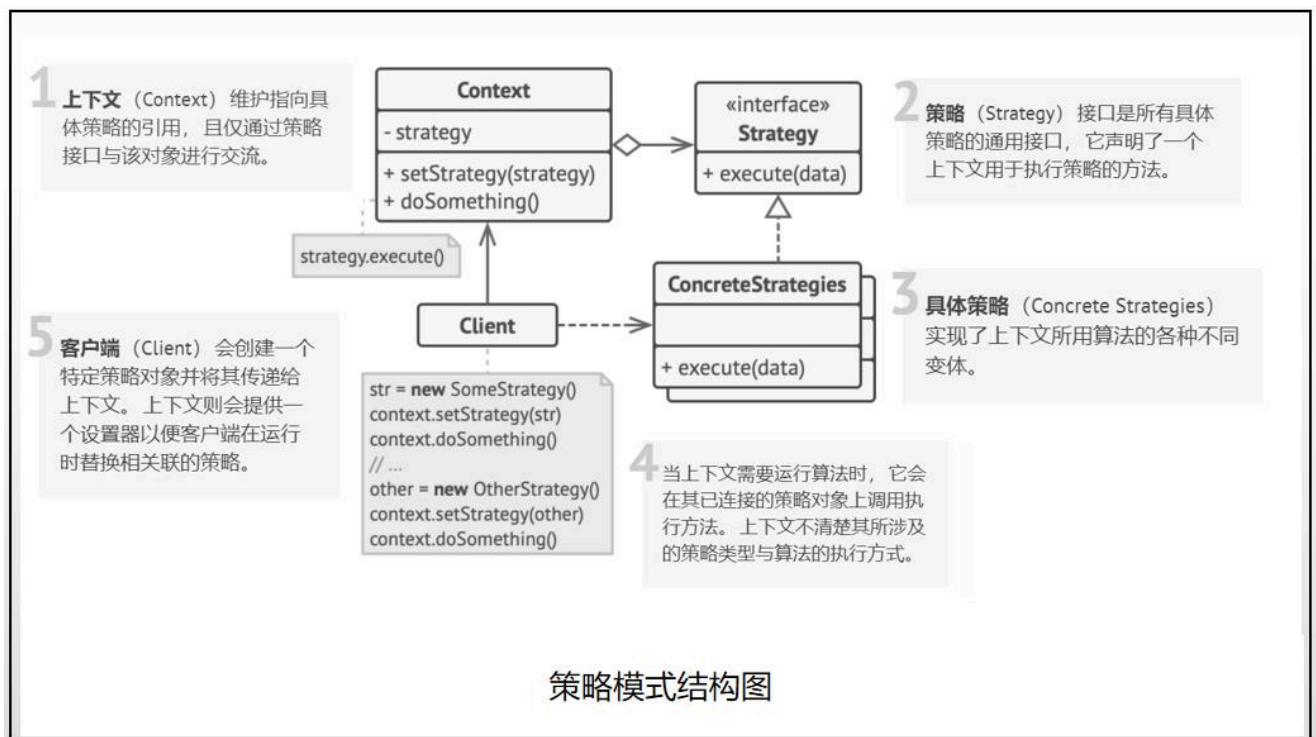
- `remove()` 移除当前返回的元素。



策略模式

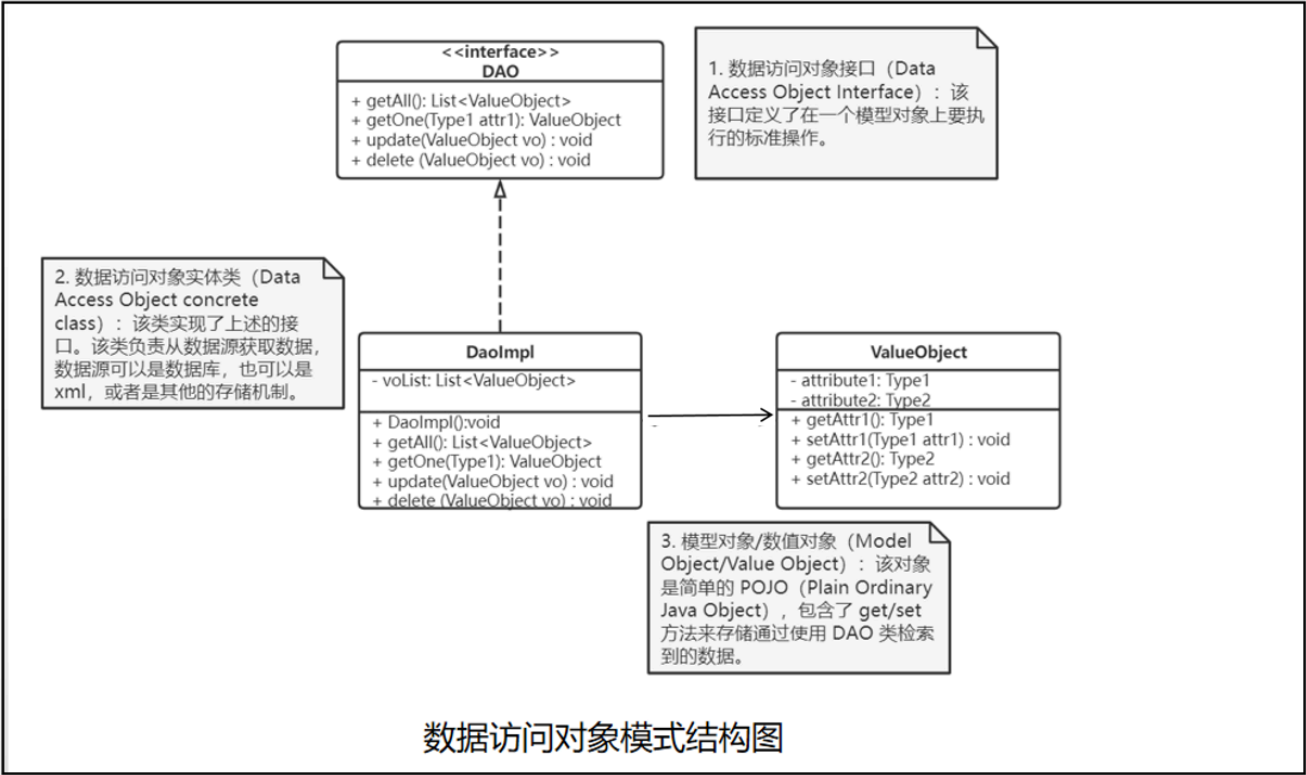
策略模式将对象与算法分离，使得在有多种算法相似的情况下，避免多个 **if-else** 语句所带来的复杂和难以维护。

使用策略模式的优点：算法可以自由切换，扩展性好；但增加暴露的策略类可能不易维护。



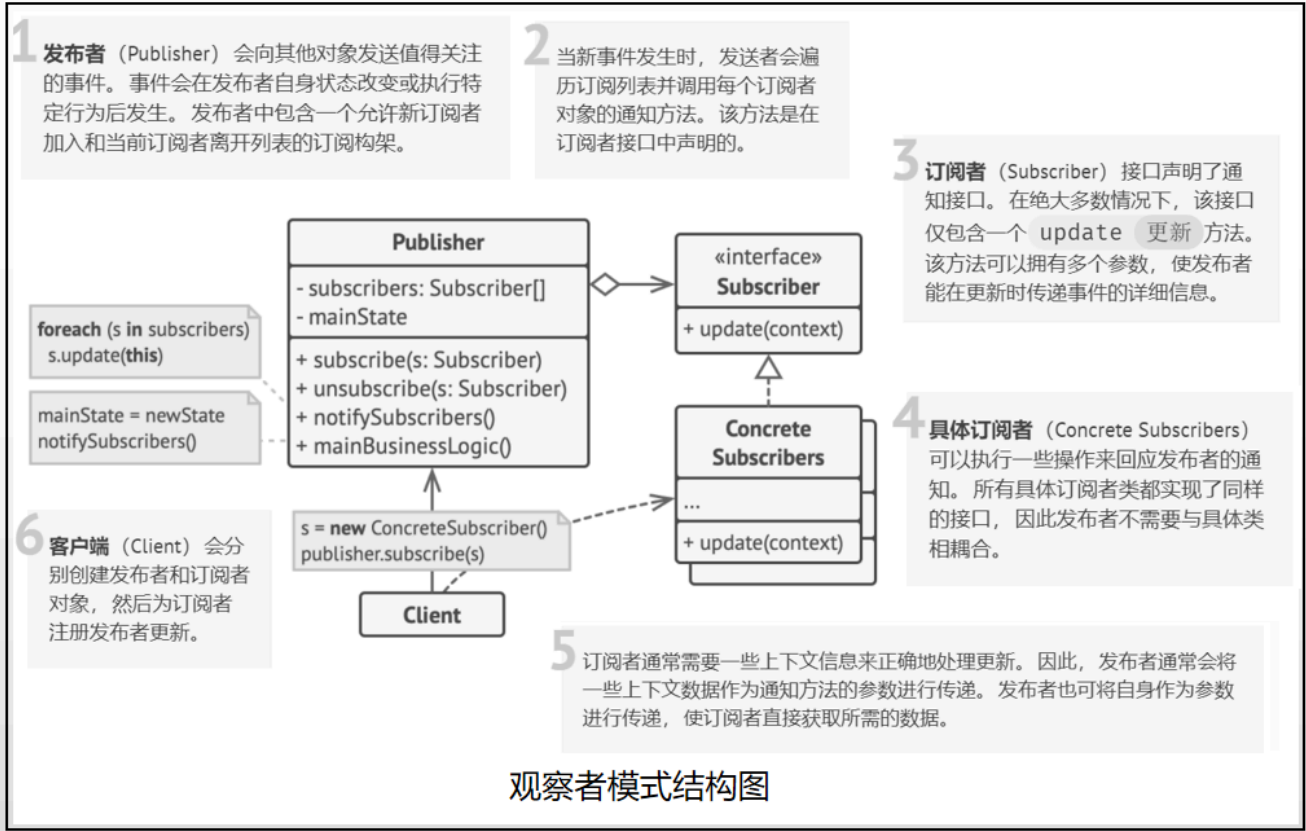
数据访问对象模式

用于将低级的数据操作从高级的业务逻辑中分离。



观察者模式

定义了对对象之间一种一对多的依赖关系, 让一个对象的改变能够影响其他对象。



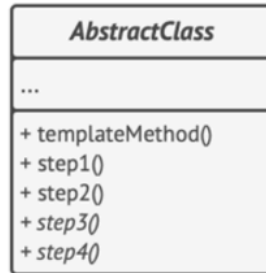
模板方法模式

定义一个操作中的算法骨架, 将一些步骤延迟到子类中, 使得子类可以不改变一个算法的具体结构即可重新定义该算法的特定步骤。

- 钩子操作（可以重新定义）
- 抽象操作（必须重新定义）

1

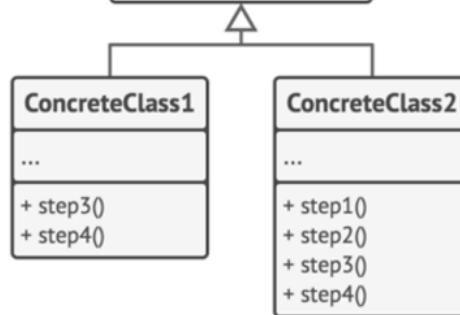
抽象类（AbstractClass）会声明作为算法步骤的方法，以及依次调用它们的实际模板方法。算法步骤可以被声明为 抽象 类型，也可以提供一些默认实现。



```
step1()
if (step2()) {
    step3()
}
else {
    step4()
}
```

2

具体类（ConcreteClass）可以重写所有步骤，但不能重写模板方法自身。



模板模式结构图